

Spring AOP

За чем скрывается магия?



Какие проблемы?

Мониторинг времени выполнения метода



Метод



Необходимо для ряда методов реализовать сбор метрики времени их выполнения и отправки этих метрик на дашборд мониторинга.

```
public Book findBook(UUID id) { no usages
    log.debug("[FIND BOOK] with id {}", id);
    long start = System.currentTimeMillis();
    try {
        return bookClient.findBookById(id);
    } finally {
        long end = System.currentTimeMillis();
        monitoringService.sendMonitoringData( method: "findBook", time: end - start);
        log.info("[FIND BOOK] Execution time = {}", end - start);
    }
}
```

Spring AOP

Аспектно-ориентированное программирование (АОП) — это парадигма программирования являющейся дальнейшим развитием процедурного и объектно-ориентированного программирования (ООП).

Идея АОП заключается в выделении так называемой **сквозной функциональности**

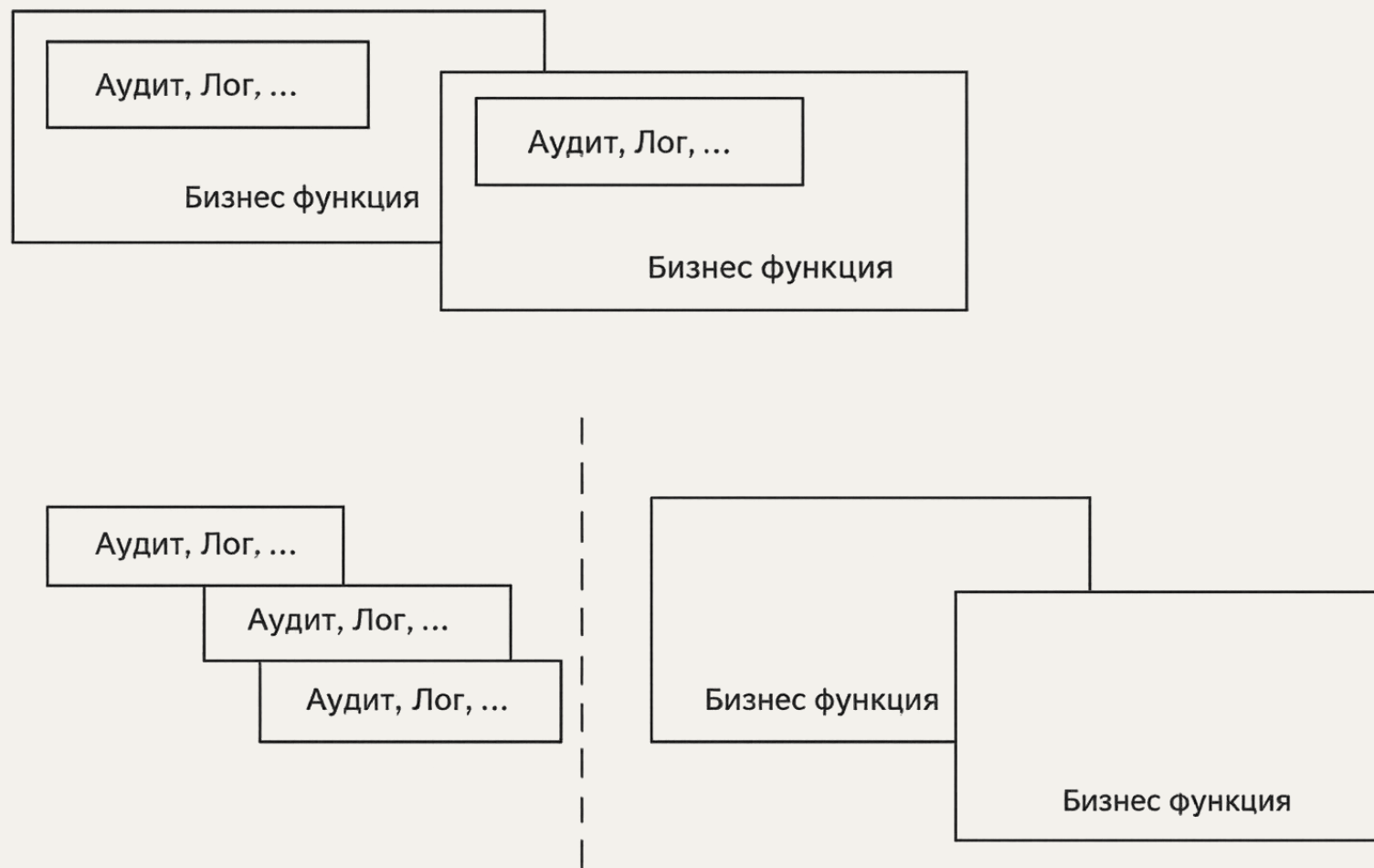
Что такое сквозная функциональ ность?

Сквозная функциональность — это поведение, которое **пересекает множество компонентов системы и повторяется в них**

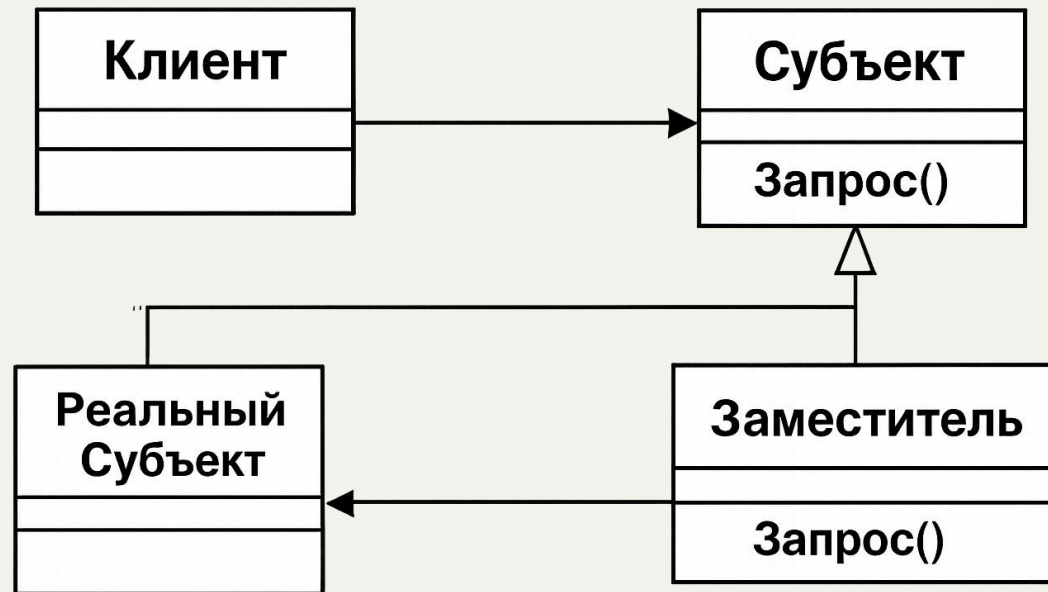
Это логика, которая часто нужна, но не является основной задачей системы



Выделение сквозной функциональности



Proxy



**БИЗНЕС
ФУНКЦИОНАЛЬНОСТЬ**

**СКВОЗНАЯ
ФУНКЦИОНАЛЬНОСТЬ**

Proxy

JDK dynamic proxy



Spring AOP по умолчанию использует JDK dynamic proxy, которые позволяют проксировать любой интерфейс (или набор интерфейсов). Если целевой объект реализует хотя бы один интерфейс, то будет использоваться динамический прокси JDK.

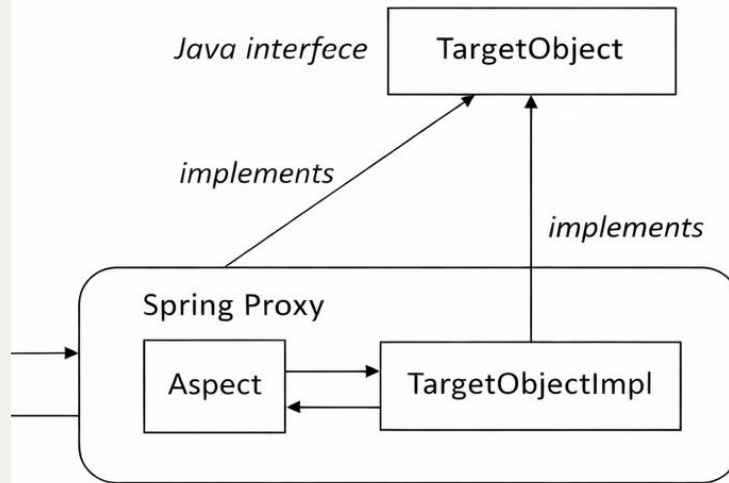
CGLIB proxy



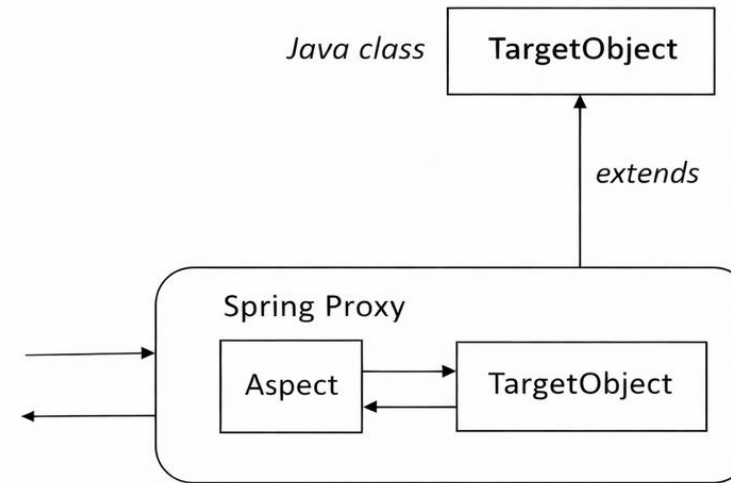
Используется по умолчанию, если бизнес-объект не реализует ни одного интерфейса.

Proxy

JDK Proxy (interface based)



CGLib Proxy (class based)



“

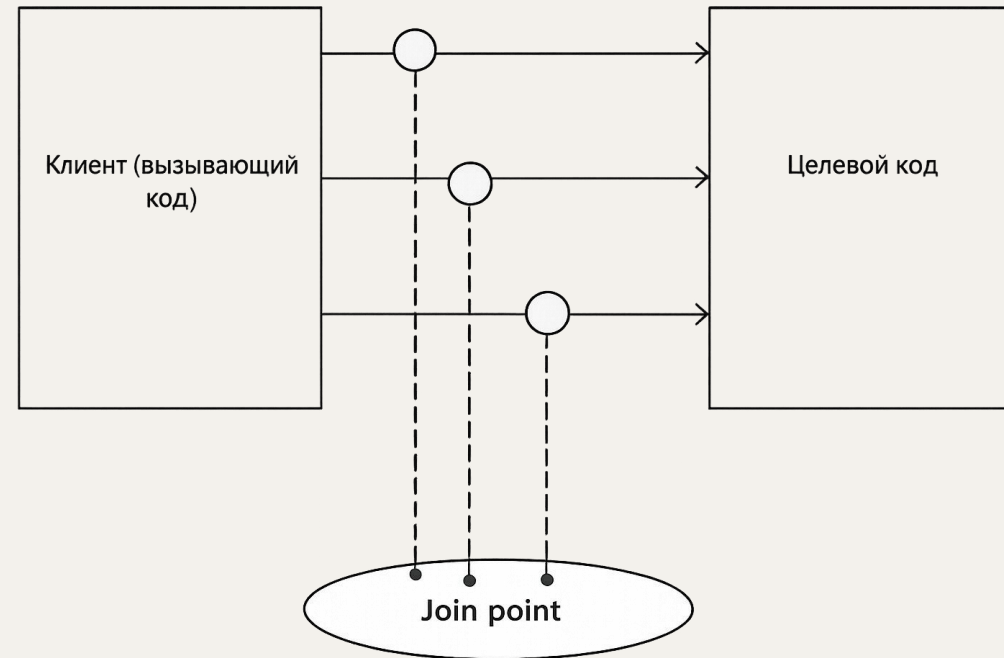
Вдохновение



— Irene M. Pepperberg

Joint Point

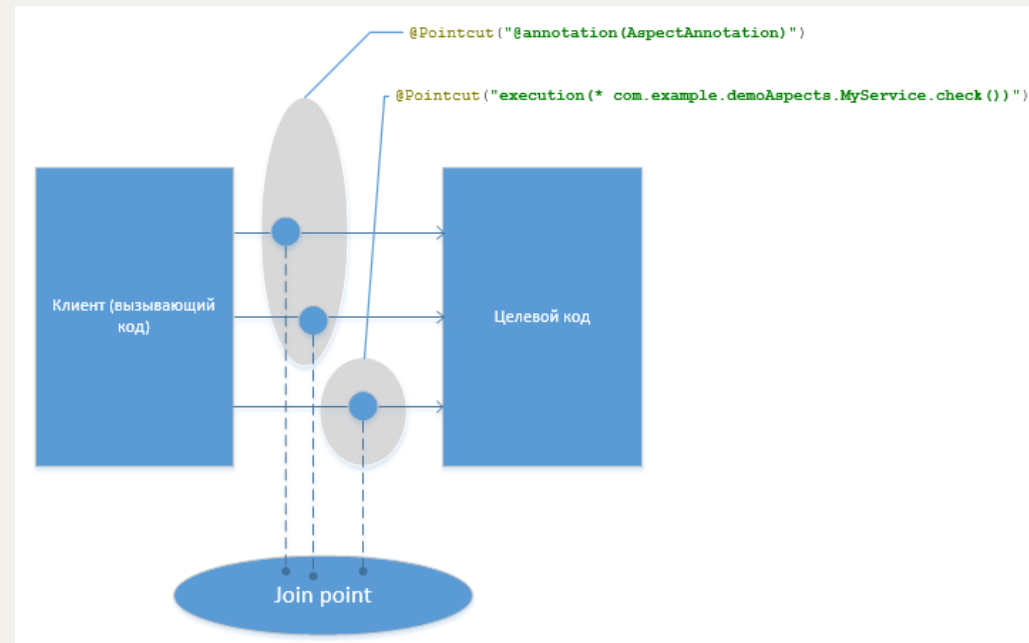
Join point — это точки наблюдения, присоединения к коду, где планируется введение функциональности.



Pointcut

Pointcut — это срез, запрос точек присоединения, — это может быть одна и более точек. Правила запросов точек разнообразные.

Правила можно объединять по &&, ||, !



Advice

Advice — набор инструкций выполняемых на точках среза (**Pointcut**)

Инструкции можно выполнять по событию разных типов:

На один **Pointcut** можно «повесить» несколько **Advice** разного типа.

01 › Before



Перед вызовом метода

02 › After



После вызова метода

03 › After returning



После возврата значения из функции

04 › After throwing



В случае **exception**

05 › After finally



В случае выполнения блока **finally**

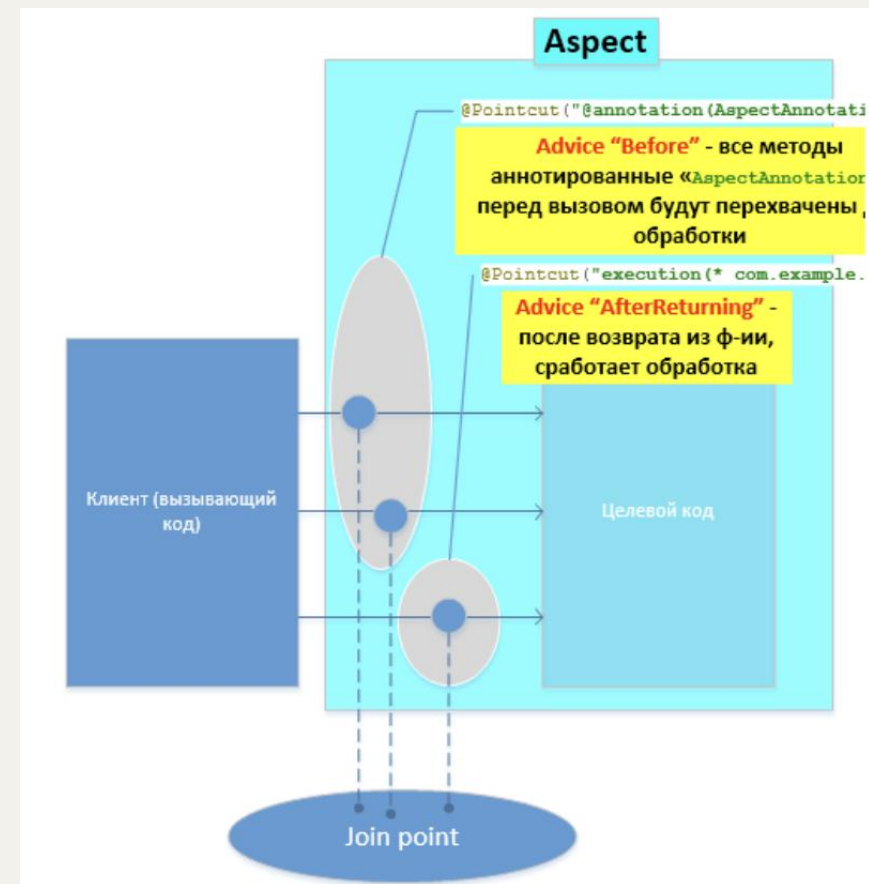
06 › Around



Можно сделать пред., пост., обработку перед вызовом метода, а также вообще обойти вызов метода

Aspect

Aspect — модуль, в котором собраны описания **Pointcut** и **Advice**.



01



```
@Service
public class MyService {

    public void method1(List<String> list) {
        list.add("method1");
        System.out.println("MyService method1 list.size=" + list.size());
    }

    @AspectAnnotation
    public void method2() {
        System.out.println("MyService method2");
    }

    public boolean check() {
        System.out.println("MyService check");
        return true;
    }
}
```



02

```

@Aspect
@Component
public class MyAspect {

    private Logger logger = LoggerFactory.getLogger(this.getClass());

    @Pointcut("execution(public * com.example.demoAspects.MyService.*(..))")
    public void callAtMyServicePublic() { }

    @Before("callAtMyServicePublic()")
    public void beforeCallAtMethod1(JoinPoint jp) {
        String args = Arrays.stream(jp.getArgs())
            .map(a -> a.toString())
            .collect(Collectors.joining(", "));
        logger.info("before " + jp.toString() + ", args=[" + args + "]);
    }

    @After("callAtMyServicePublic()")
    public void afterCallAt(JoinPoint jp) {
        logger.info("after " + jp.toString());
    }
}

```



03



```
@RunWith(SpringRunner.class)
@SpringBootTest
public class DemoAspectsApplicationTests {

    @Autowired
    private MyService service;

    @Test
    public void testLoggable() {
        List<String> list = new ArrayList();
        list.add("test");

        service.method1(list);
        service.method2();
        Assert.assertTrue(service.check());
    }
}
```



Результаты выполнения

```
2018-11-02 08:23:56.535 INFO 8152 --- [main] com.example.demoAspects.MyAspect : before execution(void com.example.demoAspects.MyService.method1(List)), args=[[test]]
MyService method1 list.size=2
2018-11-02 08:23:56.547 INFO 8152 --- [main] com.example.demoAspects.MyAspect : after execution(void com.example.demoAspects.MyService.method1(List))
2018-11-02 08:23:56.547 INFO 8152 --- [main] com.example.demoAspects.MyAspect : before execution(void com.example.demoAspects.MyService.method2()), args=[]
MyService method2
2018-11-02 08:23:56.548 INFO 8152 --- [main] com.example.demoAspects.MyAspect : after execution(void com.example.demoAspects.MyService.method2())
2018-11-02 08:23:56.548 INFO 8152 --- [main] com.example.demoAspects.MyAspect : before execution(boolean com.example.demoAspects.MyService.check()), args=[]
MyService check
2018-11-02 08:23:56.548 INFO 8152 --- [main] com.example.demoAspects.MyAspect : after execution(boolean com.example.demoAspects.MyService.check())
```

01



```
@Retention(RetentionPolicy.RUNTIME)
@Target({ElementType.TYPE, ElementType.METHOD})
public @interface Loggable {

}
```



02

```

@Aspect
@Component
public class LoggerAspect {

    @Pointcut("@annotation(Loggable)")
    public void loggableMethod() {
    }

    @Around("loggableMethod()")
    public Object logExecutionTime(ProceedingJoinPoint joinPoint) throws Throwable {

        MethodSignature methodSignature = (MethodSignature) joinPoint.getSignature();
        String className = methodSignature.getDeclaringType().getSimpleName();
        String methodName = methodSignature.getName();
        final Stopwatch stopwatch = new Stopwatch();
        stopwatch.start();
        try {
            return joinPoint.proceed();
        } finally {
            stopwatch.stop();
            System.out.println("Execution time for " + className + "." + methodName + " :: " +
stopwatch.getTotalTimeMillis() + " ms");
        }
    }
}

```



03



```
@Component
public class Pojo {

    @Loggable
    public void test(){
        System.out.println("test method called");
        this.testUtil();
    }

    @Loggable
    public void testUtil(){
        System.out.println("testUtil method called");
    }

}
```



04



```
@SpringBootApplication
public class SpringAopDemoApplication implements CommandLineRunner {

    @Autowired
    Pojo pojo;

    public static void main(String[] args) {
        SpringApplication.run(SpringAopDemoApplication.class, args);
    }

    @Override
    public void run(String... args){

        pojo.test();
        System.out.println("Out of Test");
        pojo.testUtil();
    }
}
```



Результаты выполнения

```
test method called  
testUtil method called  
Execution time for Test.test :: 18 ms  
Out of Test  
testUtil method called  
Execution time for Test.testUtil :: 0 ms
```

Последовательность выполнения

